

Study Guide: Unit 1 Python Basics

Python Domains 1–3

Directions: This study guide helps you review everything from Domains 1–3.

Work with your assigned partner(s). Talk through your thinking, compare approaches, and help each other understand the “why,” not just the “how.”

Each section includes:

- Starter ★
- Power-Up ★★
- Legendary ★★★
- Mythic ★★★★★ (new highest tier — chosen by your class!)
- Discussion prompts
- Paste-your-code boxes

Jump to section:

1. [Operator Order & Expressions](#)
2. [Data Types & Type Conversion](#)
3. [Branching \(if / elif / else\)](#)
4. [Loops \(for, while, break, continue\)](#)
5. [Nested Loops](#)
6. [File Handling \(open, read, write, append\)](#)
7. [Formatted Output \(f-strings\)](#)
8. [Indexing & Slicing \(Expanded\)](#)

SECTION 1: Operator Order & Expressions

Starter ★

Evaluate the following expression *without running it*:

None

```
value = 12 - 3 * 2
```

- A. What is the final value
- B. Which operator(s) run first
- C. Explain why

Power-Up ★★

Evaluate this expression step-by-step:

None

```
result = (15 % 4 + 2 * 3) <= 10 or not (7 - 2 * 2 == 3)
```

- A. Rewrite the expression in the order Python evaluates it
- B. Explain why Python chooses that order
- C. Predict the final value

Legendary ★★★

Determine the operator order in this expression:

None

```
value = (18 // 4 - 1) * 2 != 8 and (3 ** 2 > 7 or False)
```

- A. List the operators in the order they run
- B. Show each step of evaluation
- C. Predict the final value

Mythic ★★★★★

Create your own expression that uses:

- arithmetic
- comparison
- logical `not`, `and`, and `or`
- parentheses

Challenge your partner to evaluate it.

Paste code/screenshot here:

Discussion Prompts

- Why does `not` run before `and`?
- Why do comparison operators run after arithmetic?
- How can parentheses change everything?

SECTION 2: Data Types & Type Conversion

Starter ★

Create one variable for each type:

- string
- integer
- float
- boolean
- list

Print each variable and its type.

Paste code/screenshot here:

Power-Up ★★

Ask the user for a number.

Print:

- the number as a string
- the number as an integer
- the number as a float

Explain what happens if the user enters "3.5".

Paste code/screenshot here:

Legendary ★★★

Write a program that:

1. Asks the user for two values
2. Detects whether each value is numeric or text
3. Converts numeric values to integers or floats appropriately
4. Prints a summary of what types were detected

Paste code/screenshot here:

Mythic ★★★★★

Build a tiny "type analyzer" that:

- takes any input
- guesses the type
- converts it if possible
- prints the converted value and type

Paste code/screenshot here:

Discussion Prompts

- Why does `input()` always return a string?
- When is it better to convert to `float` instead of `int`?
- What kinds of errors happen when conversion fails?

SECTION 3: Branching (`if` / `elif` / `else`)

Starter ★

Write a program that asks for a number and prints:

- "Positive"
- "Negative"
- "Zero"

Paste code/screenshot here:

Power-Up ★★

Ask the user for two numbers.

Use:

- `if`
- `elif`
- `else`
- one compound condition (`and` or `or`)

Print something meaningful about the numbers.

Paste code/screenshot here:

Legendary ★★★

Write a branching system that classifies a score:

- 90+ → "S-Tier"
- 80-89 → "A-Tier"
- 70-79 → "B-Tier"
- 60-69 → "C-Tier"
- below 60 → "Try Again"

Use at least one compound condition.

Paste code/screenshot here:

Mythic ★★★★★

Create a branching system that behaves differently depending on:

- the user's age

- the time of day
- whether they have a “passcode”

Paste code/screenshot here:

Discussion Prompts

- When is `elif` better than multiple `if` statements?
- What makes a compound condition useful?
- How can branching help in game programming?

SECTION 4: Loops (for, while, break, continue)

Starter ★

Write a loop that prints numbers 1–10.

Paste code/screenshot here:

Power-Up ★★

Create a list of 5 items.

Loop through it and:

- skip one item using `continue`
- stop early using `break`
- print the items that are not skipped or stopped

Paste code/screenshot here:

Legendary ★★★

Write a loop that:

- asks the user for input repeatedly
- stops only when the user types `"quit"`
- ignores blank input using `continue`

Paste code/screenshot here:

Mythic ★★★★★

Create a loop that simulates a simple game mechanic:

- player starts with 10 HP
- each loop, HP decreases randomly
- loop ends when $HP \leq 0$

- print a “game over” message

Paste code/screenshot here:

Discussion Prompts

- When is a `while` loop better than a `for` loop
- Why is `break` useful
- How do loops show up in games

SECTION 5: Nested Loops

Starter ★

Print a 3×5 rectangle of `*`.

Paste code/screenshot here:

Power-Up ★★

Ask the user for a number.

Print a triangle of that height.

Paste code/screenshot here:

Legendary ★★★

Print a multiplication table (1–10).

Paste code/screenshot here:

Mythic ★★★★★

Create a pattern where:

- even rows use one symbol
- odd rows use another
- size is chosen by the user

Paste code/screenshot here:

Discussion Prompts

- Why do nested loops matter in game programming
- What kinds of patterns can nested loops generate

- How do nested loops relate to grids, maps, or levels

SECTION 6: File Handling (open, read, write, append)

Starter ★

Create a file and write two lines to it.

Paste code/screenshot here:

Power-Up ★★

Ask the user for a filename.

If it exists:

- read it
- print the contents

If not:

- create it
- write two lines
- print "File created"

Paste code/screenshot here:

Legendary ★★★

Write a file analyzer that prints:

- number of lines
- number of characters
- whether the file contains the word "level"

Paste code/screenshot here:

Mythic ★★★★★

Create a "save file" system:

- ask the user for a username
- create a file named after them
- store stats (HP, XP, level)
- allow them to "load" the file later

Paste code/screenshot here:

Discussion Prompts

- Why is file handling important in games
- What's the difference between write and append

- Why is checking file existence useful

SECTION 7: Formatted Output (f-strings)

Starter ★

Ask the user for their name and print:

None

```
Hello, <name>!
```

Paste code/screenshot here:

Power-Up ★★

Ask the user for:

- name
- level
- favorite item

Print a formatted message using an f-string.

Paste code/screenshot here:

Legendary ★★★

Print a multi-line message using:

- f-strings
- `\n`
- `.format()`

Paste code/screenshot here:

Mythic ★★★★★

Create a “character card” that prints:

- name
- class
- level
- HP
- XP
- special ability

Format it cleanly using f-strings.

Paste code/screenshot here:

SECTION 8: Indexing & Slicing (Expanded)

Starter ★

Given:

```
Python
code = "PYTHON-LEVEL-UP"
```

Write the slice that returns:

1. "PYTHON"
2. "LEVEL"
3. "UP"
4. "THON"
5. "P-L-U" (use a step)

Paste code/screenshot here:

Power-Up ★★

Given:

```
Python
serial = "A1B2C3D4E5F6"
```

Write slices that return:

1. the last 4 characters
2. everything except the first 2 characters
3. every second character
4. the middle 3 characters

Paste code/screenshot here:

Legendary ★★★

Write your own string (at least 10 characters).

Create five slices that show:

- first 3 characters
- last 3 characters
- a middle section
- a slice using a negative start
- a slice using a step

Paste code/screenshot here:

Mythic ★★★★★

Decode this message using slicing:

None

```
msg = "X0Y1Z2A3B4C5D6"
```

Extract only the letters.

Paste code/screenshot here:

Discussion Prompts

- Why is slicing useful in game programming
- How does slicing help with parsing commands or player input
- What's the difference between indexing and slicing

FINAL DISCUSSION (Partner/Trio)

Talk through these questions together:

- Which concepts feel strongest for you right now
- Which ones do you want to practice more before the exam
- How can you support each other during the exam
- What strategies help you debug or understand code